

SUBJECT: DATA COMMUNICATION

# Cyclic Redundancy Check (CRC) Calculator



Reliable error detection via binary polynomial division

Aryan Nanda (Roll No. 16) | Sai Yadav (Roll No. 20)

Faculty Guide: Prof. Rohini Khalkar

Bharati Vidyapeeth College of Engineering

# Introduction

## 1 What is CRC?

Error detection based on binary polynomial division.

## 2 Why error detection?

Data can change while moving through a communication channel.

## 3 Basic idea of CRC

Generate a remainder, transmit it with the data, then verify at the receiver.

## 4 Where CRC is used

Computer networks, Ethernet, storage and digital communication.

# Problem Statement

Manual CRC calculation using binary division is time-consuming and prone to errors. Therefore, a software-based CRC calculator is developed to automatically generate and verify CRC values.

Time-consuming

Error-prone

Solution: C++ calculator

# Objectives

What the calculator is built to do

## 1 Calculate CRC

Remainder from data and generator, automatically.

## 2 Modulo-2 division

Carry-free division performed with XOR.

## 3 Generate transmitted data

Builds the transmitted data = data + CRC.

## 4 Verify received data

Divides the received frame by the same generator.

## 5 Detect transmission errors

Non-zero remainder means the frame is corrupted.

## 6 Easy-to-use interface

Menu-driven console with input validation.

# Working / Principle of CRC

Six-stage sender pipeline that makes the codeword self-checking



## MATHEMATICAL IDENTITY

$$T(x) = D(x) * x^{(n-1)} + R(x)$$

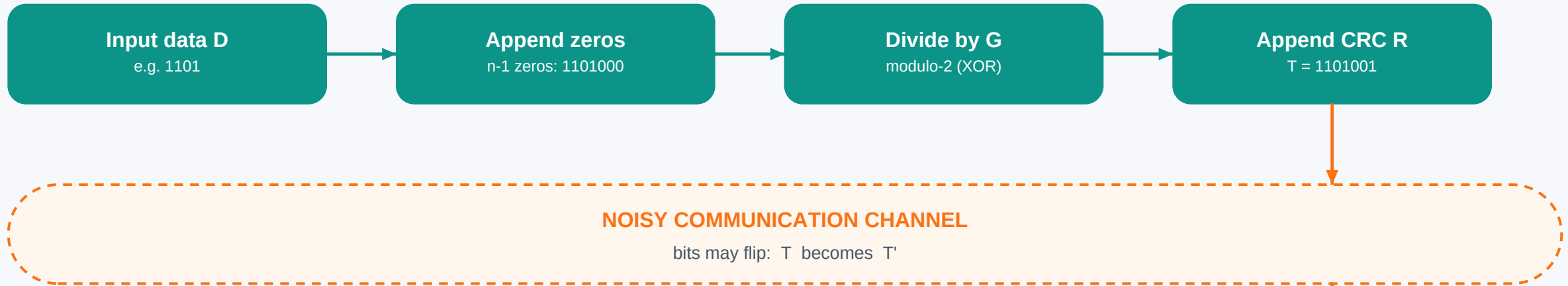
## RECEIVER PROPERTY

$$T(x) \bmod G(x) = 0 \quad (\text{valid frame})$$

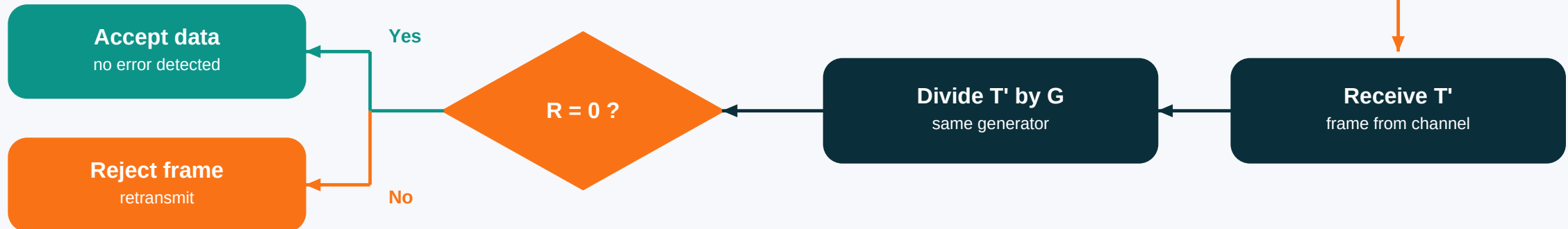
# Sender-receiver block diagram

How a CRC-protected frame travels through a noisy channel

## SENDER



## RECEIVER



# CRC Algorithm

Worked example: data 1101, generator 1011 (n = 4)

- 1 Enter data bits: 1101
- 2 Enter generator polynomial: 1011 (n = 4)
- 3 Append n-1 = 3 zeros: 1101000
- 4 Perform modulo-2 division using XOR
- 5 Obtain the CRC remainder: 001
- 6 Append remainder to data: 1101001
- 7 Verify at receiver: remainder 000 means valid

**DIVIDEND 1101000 / GENERATOR 1011**

| STEP | WINDOW | XOR  | NEXT |
|------|--------|------|------|
| 1    | 1101   | 0110 | 1100 |
| 2    | 1100   | 0111 | 1110 |
| 3    | 1110   | 0101 | 1010 |
| 4    | 1010   | 0001 | END  |

**Remainder = 001**

# System Requirements

Standard C++ keeps execution lightweight and portable

**Software**

C++

**Compiler**

C++ compiler or online compiler (e.g. Programiz)

**Operating system**

Windows or Linux



# Challenges Faced and Solutions

## CHALLENGE

## SOLUTION

Modulo-2 (XOR) division has no carries or borrows

Implemented xorOperation() on bit strings and checked against hand calculation.

Invalid inputs such as non-binary digits or a bad generator

isBinary() validation; generator must start with 1 and not exceed the data length.

Weak generators missed errors (e.g. 10000 only checks the last 4 bits)

Used proper generators like 1011 and tested by flipping single bits.

Sender and receiver logic in one program

Menu-driven design with separate generate() and verify() functions.

# Future Scope

## 1 Standard polynomials

Support CRC-8, CRC-16 and CRC-32 generators.

## 2 Graphical interface

A GUI with buttons instead of a console menu.

## 3 Step-by-step view

Show each XOR step of the division for teaching.

## 4 Burst-error testing

Test and report how many error patterns are detected.

## 5 Frame and file input

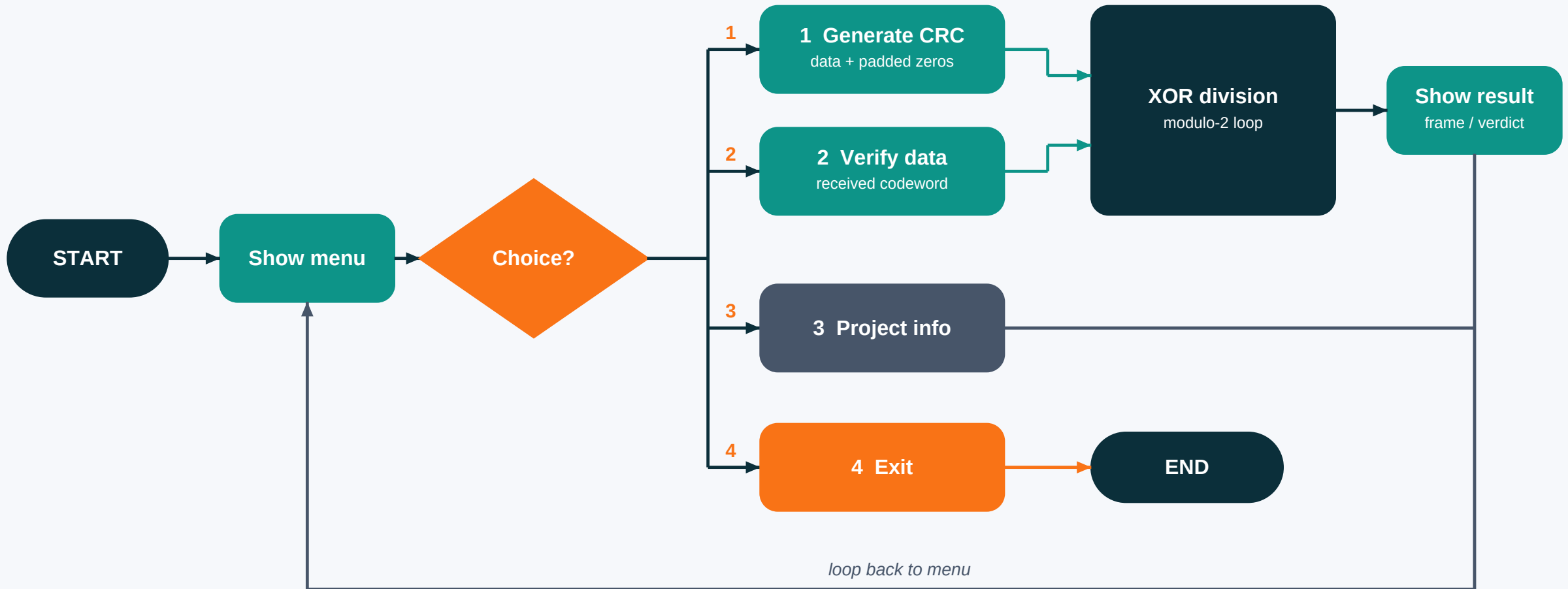
Compute CRC for whole frames or files.

## 6 Hardware style (LFSR)

Model the shift-register circuit used in real devices.

# System Design / Flowchart

Menu-driven program that loops until the user chooses Exit



*Invalid input: error message, then back to menu*

# Implementation

Language: C++ | Functions used

**isBinary()**

Validates that an input contains only binary digits.

**xorOperation()**

Applies XOR to two bit strings.

**divideCRC()**

Performs modulo-2 division.

**generateCRC()**

Calculates the remainder.

**generate()**

Prepares the transmitted data.

**verify()**

Checks received data for errors.

# Output / Screenshots

Console output of the C++ program (data 1101, generator 1011)

## 1. CRC GENERATION

|               |         |
|---------------|---------|
| Data          | 1101    |
| Generator     | 1011    |
| Augmented     | 1101000 |
| CRC Remainder | 001     |

## 2. TRANSMITTED DATA

|           |   |   |   |         |   |   |
|-----------|---|---|---|---------|---|---|
| 1         | 1 | 0 | 1 | 0       | 0 | 1 |
| data 1101 |   |   |   | CRC 001 |   |   |

Transmitted Data : 1101001

## 3. NO ERROR DETECTED

|               |                   |
|---------------|-------------------|
| Received Data | 1101001           |
| Remainder     | 000               |
| Result        | NO ERROR DETECTED |

## 4. ERROR DETECTED

|               |                |
|---------------|----------------|
| Received Data | 1100001        |
| Remainder     | 011 (non-zero) |
| Result        | ERROR DETECTED |

# Conclusion

What this project demonstrates

## 1 Reliable detection

CRC detects transmission errors using modulo-2 (XOR) division by an agreed generator.

## 2 Automation

The calculator replaces slow, error-prone manual division with instant, repeatable results.

## 3 Tested result

1101 with generator 1011 gives frame 1101001 (accepted); 1100001 gives remainder 011 (rejected).

## 4 Clean design

Modular, menu-driven C++ with input validation and no external libraries.

# Thank You

Questions?

---

Aryan Nanda (Roll No. 16) | Sai Yadav (Roll No. 20)

Faculty Guide: Prof. Rohini Khalkar

Bharati Vidyapeeth College of Engineering